

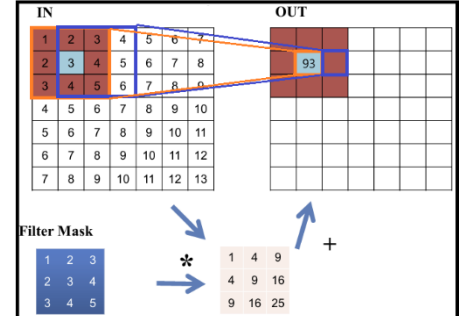
HW4: Implementing Direct Convolution & im2col+GEMM using CUDA

1. 목표: CUDA를 이용하여 Direct Convolution, im2col, GEMM을 구현 후 Optimization을 진행한다.

2. What is 2D Convolution

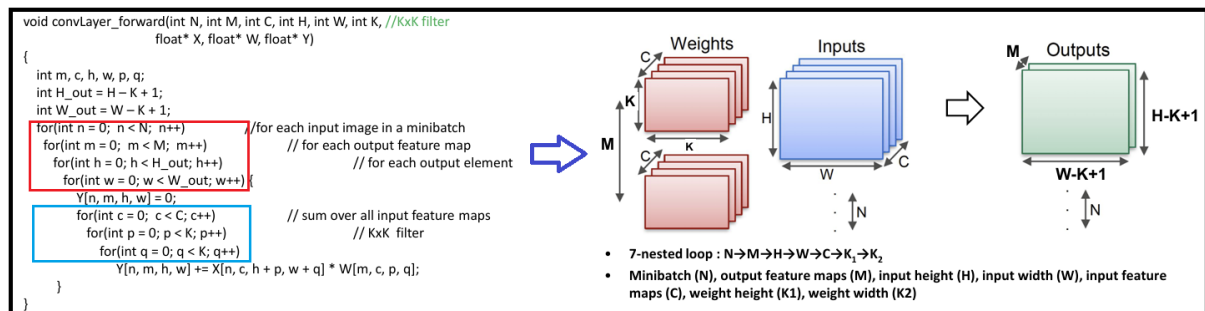
2D Convolution은 입력 행렬 I (크기 $H \times W$)와 Filter(Kernel) K (SIZE: $F \times F$)를 사용하여 출력 행렬 O 를 생성하는 연산이다. 출력의 각 elements는 다음과 같은 수식으로 계산된다.

$$O(i, j) = \sum_{m=0}^{F-1} \sum_{n=0}^{F-1} I(i+m, j+n) \cdot K(m, n)$$



- i, j : 출력 행렬의 위치 index
- m, n : Filter 의 위치 index
- $I(i+m, j+n)$: input 행렬에서 Filter 와 겹치는 부분의 값
- $K(m, n)$: Filter 값

3. What is Direct Convolution?



Direct Convolution이란, filter를 직접 input data에 적용하는 방식으로 연산을 수행하는 가장 직관적인 convolution 연산 방법이다.

1) 먼저, 기본적인 multi-channel 2D convolution 연산에서 쓰이는 변수들을 알아본다.

N : input data의 개수, M 은 filter의 개수, C 는 input channel의 수(ex: RGB image면 3), H 는 input data의 height, W 는 input data의 width, K 는 filter의 SIZE이다.

Padding 이 없고 Stride 가 1 인 경우, 출력 크기는 $H_{out} = H - K + 1$, $W_{out} = W - K + 1$ 로 계산된다.

2) 이제 Direct Convolution이 무엇을 계산하는지 알아본다.

CNN에서 2D Convolution은 각 input image에 대하여 M 개의 Filter를 적용하여 M 개의 Output image를 생성한다. 이를 input image개수만큼 반복하기에 M 개의 set를 가지는 output image를 총 N 개 생성한다.

이때, input image 하나와 Filter 하나는 아래의 수식으로 계산된다.

$$Y[n, m, h, w] = \sum_{c=0}^{C-1} \sum_{p=0}^{K-1} \sum_{q=0}^{K-1} X[n, c, h + p, w + q] \cdot W[m, c, p, q]$$

따라서, 위 수식을 지속적으로 반복하여 M개의 set를 가지는 output image를 총 N개 생성하는 방법을 Direct Convolution이라고 한다.

3-1. How can we implement Direct Convolution by using CUDA Programming model?

<pre>__global__ void conv2d_direct_kernel(const float* __restrict__ x, const float* __restrict__ w, float* __restrict__ y, int N, int C, int H, int W, int K_out, int kH, int kW, int H_out, int W_out) { const int h_out = blockIdx.y * BLOCK_SIZE + threadIdx.y; const int w_out = blockIdx.x * BLOCK_SIZE + threadIdx.x; const int nK = blockIdx.z; // fused (n, k) dimension const int n = nK / K_out; // Batch index, input image index const int k = nK % K_out; // Output channel index, filter index; float acc = 0.f; // all input channels & accumulation for (int c = 0; c < C; ++c) { for (int p = 0; p < kH; ++p) { const int h_in = h_out + p; for (int q = 0; q < kW; ++q) { const int w_in = w_out + q; acc += x[(n * C + c) * H + h_in] * w[w_out + w_in] * w[((k * C + c) * kH + p) * kW + q]; } } } y[(n * K_out + k) * H_out + h_out] * W_out + w_out] = acc; }</pre>	<pre>void launch_conv2d_direct(const float* x, const float* w, float* y, int N, int C, int H, int W, int K, int kH, int kW) { const int H_out = H - kH + 1; const int W_out = W - kW + 1; dim3 block(BLOCK_SIZE, BLOCK_SIZE); dim3 grid(W_out / BLOCK_SIZE, H_out / BLOCK_SIZE, N * K); // merge (n,k) dimension conv2d_direct_kernel<<grid, block>>>(x, w, y, N, C, H, W, K, kH, kW, padH, padW, strideH, strideW, dilH, dilW, H_out, W_out); }</pre>
---	--

CUDA Programming의 경우 각 thread는 $Y[n, m, h, w] = \sum_{c=0}^{C-1} \sum_{p=0}^{K-1} \sum_{q=0}^{K-1} X[n, c, h + p, w + q] \cdot W[m, c, p, q]$ 만 계산하도록 하고, n, m, h, w는 thread의 idx를 이용하여 계산한다.

1) h_out: output Matrix의 row index로 gridDim.y를 H_out / BLOCK_SIZE와 같이 지정 후, blockIdx.y * BLOCK_SIZE + threadIdx.y 로 계산 가능하다.

2) w_out: output Matrix의 column index로 h_out과 마찬가지로 유사하게 계산 가능하다.

3) n: Input Image의 index로 grid를 생성 시 z dimension을 n*k와 같이 지정하였기에, K_out으로 나눠 계산 가능하다. K_out은 filter의 개수이다.

4) k: Filter의 index이자, Output Channel의 index로 z dimension을 n*k와 같이 지정하였기에, K_out으로 나눈 나머지로 계산 가능하다.

위와 같이 구현을 하게 되면 각각의 thread는 Output Matrix y의 element 1개를 담당하여 Parallel 하게 연산이 진행된다.

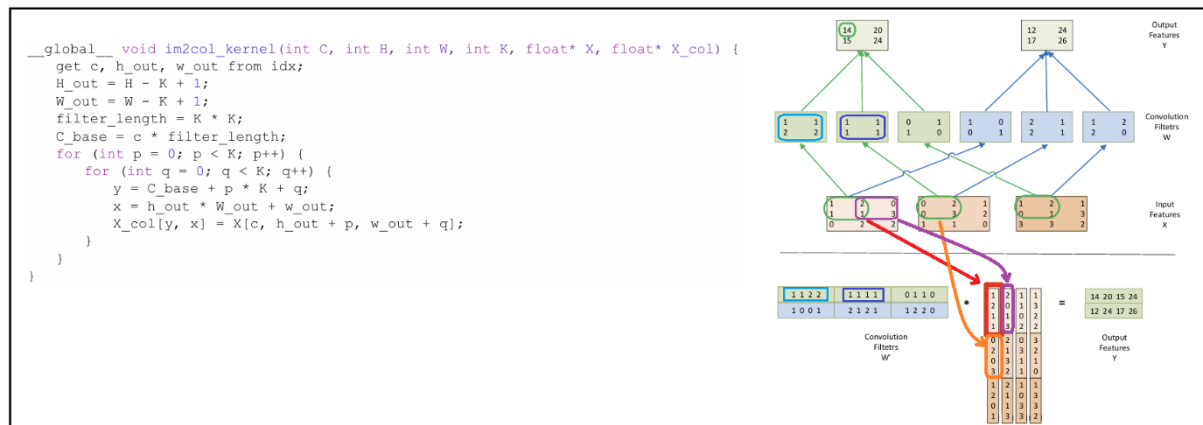
4. What is im2col?

im2col은 input image를 convolution 연산에 적합한 column 형태로 재배치하는 기법이다. im2col은 앞서 살펴보았던 Direct Convolution을 Matrix Multiplication으로 변환하여 Convolution을 연산할 수 있게 한다.

4-1. why we are using im2col?

Direct convolution은 Filter를 input data에 sliding window 방식으로 순차적으로 연산하여 output Matrix를 생성하지만, 이는 반복적인 메모리 접근과 계산이 많아 비효율적일 수 있다. 더불어, GPU는 Parallel 컴퓨팅에 최적화되어 있으며, 특히 GEMM 연산을 빠르게 처리할 수 있다. 따라서, Convolution연산을 GEMM으로 바꿔서 연산을 진행한다면 GPU의 빠른 속도를 최대한 활용할 수 있기에 im2col을 사용하여 GEMM연산을 통한 Convolution연산이 가능하도록 바꾼다.

4-2. How the im2col is implemented?



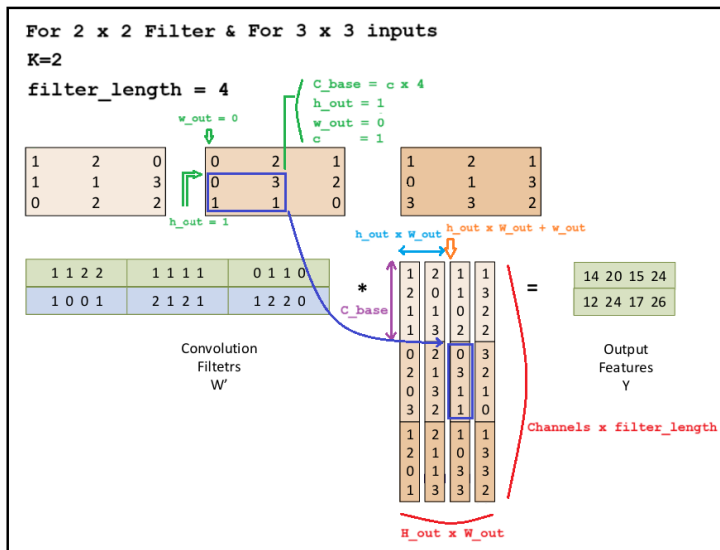
im2col은 input data에서 filter가 적용될 각 window에 해당하는 data를 추출하여 column 형태로 재배치한다.

재배치 시 각 window와 Filter에 대한 MAC연산을 vector x vector연산으로 바꾼다. 이를 반복하게 되면 결국, Matrix x Matrix연산을 수행 시 2D Convolution을 한 것과 같은 결과를 도출한다.

위에 제시된 CUDA Programming code는 기본적인 im2col 동작을 진행한다.

- C, H, W: input data의 channel 수, height, width.
- K: Filter 크기(K×K).
- X: input data Matrix.
- X_col: im2col로 변환된 output Matrix.
- H_out, W_out: output Matrix의 height, width.
- filter_length = 하나의 filter를 flatten 했을 때의 길이 (= K * K).
- C_base = c * filter_length: 현재 channel의 시작 위치.

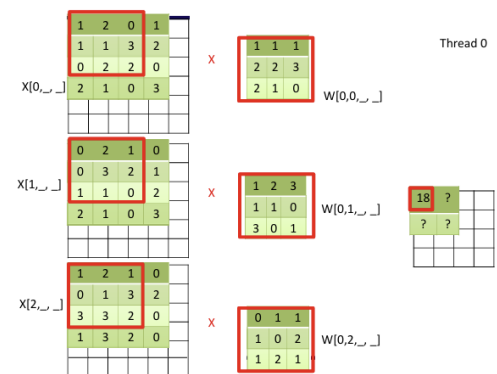
위 코드의 동작을 그림으로 나타내면 아래 그림과 같다.



5. 기존 Direct Convolution에서 최적화할 수 있는 points

1) Using Shared Memory

앞서 소개하였던 기존 code를 이용하면 3 x 3 Window와 3 x 3 Filter가 MAC 연산이 될 때마다, X Matrix를 global memory에서 읽고, W Filter를 global memory에서 읽는다. 하지만, 오른쪽 그림을 보면, X Matrix에서는 연산마다 겹치는 elements가 많고, Filter는 항상 같은 elements를 사용한다. 따라서, shared memory를 사용할 경우 겹치는 elements에 대한 reuse가 증가하여 performance가 상승할 것으로 예상된다.



2) Considering Coalescing

오른쪽의 기존 code를 보게 되면, 빨간색 네모 박스를 친 X Matrix에서 data를 load하여 연산하는 과정은 tx가 연속적으로 X Matrix의 element를 접근하므로 coalescing이 되어 있다. 더불어, y에 연산결과를 store하는 과정 또한 연속된 memory address를 tx가 접근하고 있기에 coalescing이다. 따라서, 이 coalescing에 관하여는 잘 수행되고 있는 것으로 판단된다.

```
const int h_out = blockIdx.y * BLOCK_SIZE + threadIdx.y;
const int w_out = blockIdx.x * BLOCK_SIZE + threadIdx.x;
float acc = 0.f;
// all input channels & accumulation
for (int c = 0; c < C; ++c) {
    for (int p = 0; p < kW; ++p) {
        const int h_in = h_out + p;
        for (int q = 0; q < kW; ++q) {
            const int w_in = w_out + q;
            acc += x[(n * C + c) * H + h_in] * W[w_in] * w[(k * C + c) * kW + p] * q;
        }
    }
}
y[(n * K_out + k) * H_out + h_out] * W_out[w_out] = acc;
```

3) Considering Bank Conflict

X Matrix와 W Filter 모두를 Shared Memory에 store후 사용하게 된다면, Shared Memory에서 같은 Bank의 다른 Data를 access하려는 경우가 많을 것이다. 특히, 모든 thread에서 항상 같은 Filter를 access하기 때문에 Filter에서 같은 Bank에 대한 access는 매우 많을 것으로 예상된다. 따라서, 이러한 conflict문제를 padding기법을 이용하여 방지 시 performance가 향상될 것으로 예상된다.

4) 위의 optimization 기법을 적용 시 예상되는 성능

For 3 x 3 Filter & TILE_SIZE = 16

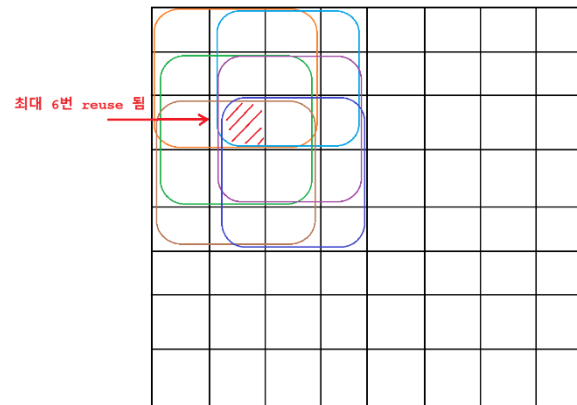
가정 1: shared memory 2 cycles access time / global memory 200 cycles access time

가정 2: for naïve implementation, total memory access time = total compute time

3 x 3 Filter & TILE_SIZE = 16의 경우

위와 같이 가정을 하고 대략적으로 모든 element가 6번 reuse 된다고 근사를 하면, $Speed\ up = \frac{1}{0.5 + \frac{0.5}{5.7}} = 1.7$ 이 예상된다.

(그림에서는 16 x 16 TILE 중 4 x 4 부분만 자름)



For 7 x 7 Filter & TILE_SIZE = 16

위와 같은 가정을 진행하고, 7 x 7 Filter & TILE_SIZE = 16의 경우

Element의 reuse는 최대 14번 발생할 수 있다.

위와 같이 가정을 하고 대략적으로 모든 element가 14번 reuse된다고 근사를 하면,

$Speed\ up = \frac{1}{0.5 + \frac{0.5}{12.39}} = 1.85$ 가 예상된다.

6. 기존 IM2COL에서 최적화할 수 있는 points

1) Using Shared Memory

2D convolution 연산 시 Filter가 input Matrix를 row-wise로 지나가며 연산이 된다. IM2COL은 이 경로의 2D window를 1D Vector(= column)으로 만드는 과정이다. Direct Convolution에서 보았듯이 Filter가 input Matrix를 지나는 경로에는 겹치는 elements가 많다. 따라서, 겹치는 elements를 shared memory를 이용하여 reuse를 하게 된다면 Performance 향상이 이루어질 것으로 예상된다.

2) Considering Coalescing

오른쪽 그림은 앞서 설명한 pseudo code를 C++로 나타낸 code이다. 기존 code의 빨간 네모 박스를 보게 되면 Coalescing의 경우에는 load할 때와 store할 때 모두 tx가 연속된 memory Address를 access하는 것을 볼 수 있다. 따라서, Coalescing의 관점에서는 최적화가 필요 없어 보인다.

```
int idx = blockIdx.x * blockDim.x + threadIdx.x;
int total_output_positions = N * H_out * W_out;
int n = idx / (H_out * W_out);
int h_out = (idx / W_out) % H_out;
int w_out = idx % W_out;

for (int c = 0; c < C; c++) {
    for (int p = 0; p < kH; p++) {
        for (int q = 0; q < kW; q++) {
            int h_idx = h_out + p;
            int w_idx = w_out + q;
            int filter_length = kH * kW;
            int C_base = c * filter_length;
            int y = C_base + p * kW + q;
            int x = idx;

            int x_offset = n * (C * H * W) + c * (H * W) + h_idx * W + w_idx;
            X_col[y * total_output_positions + x] = X[x_offset];
        }
    }
}
```

3) 위의 optimization 기법을 적용 시 예상되는 성능

For 3 x 3 Filter & TILE_SIZE = 16

가정 1: shared memory 2 cycles access time / global memory 200 cycles access time

가정 2: Im2Col는 대부분의 동작이 Mem to Mem로 옮기는 동작이기에 compute는 하지 않는다.

[실제로는 indexing을 조정하기 위한 약간의 compute를 하긴 한다.]

3 x 3 Filter & TILE_SIZE = 16의 경우

위와 같이 가정을 하고 대략적으로 모든 element가 6번 reuse된다고 근사를 하면, $Speed\ up = \frac{1}{\frac{1}{5.7}} = 5.7$ 이 예상된다.

For 7 x 7 Filter & TILE_SIZE = 16

위와 같은 가정을 진행하고, 7 x 7 Filter & TILE_SIZE = 16의 경우

Element의 reuse는 최대 14번 발생할 수 있다.

위와 같이 가정을 하고 대략적으로 모든 element가 14번 reuse된다고 근사를 하면,

$Speed\ up = \frac{1}{\frac{1}{12.39}} = 12.39$ 가 예상된다.

```
const int H_out = (H + 2 * padH - dilH * (kH - 1) - 1) / strideH + 1;
const int W_out = (W + 2 * padW - dilW * (kW - 1) - 1) / strideW + 1;

dim3 block(TILE_CONV2D_X, TILE_CONV2D_Y);
dim3 grid((H_out + TILE_CONV2D_X - kW - 1) / (TILE_CONV2D_X - kW + 1),
          (W_out + TILE_CONV2D_Y - kH - 1) / (TILE_CONV2D_Y - kH + 1),
          N * K);

conv2d_direct_kernel_shared<<<grid, block>>>(x, w, y,
```

겹치는 부분을 고려하였을 때, TILE Load는 TILE의 모든 data를 load하는 방식으로 진행하고, compute는 TILE의 elements 중 TILE_SIZE- kH 와 TILE_SIZE- kW에 존재하는 elements를 기준으로만 MAC연산을 진행한다.

오른쪽 code를 살펴본다.

1) FILTER SIZE를 고려하여 TILE을 겹치게 구성

초록색 네모 박스의 code: TILE을 shared memory에 load 시 FILTER로 인해 겹치는 부분을 중복하여 load할 수 있도록 한다.

2) TILE을 shared memory에 load

주황색 네모 박스의 code: ty와 tx를 모두 활용하여 TILE SIZE만큼의 elements를 모두 shared memory에 load한다.

3) shared memory의 요소를 이용하여 2D convolution 계산

오른쪽 그림과 같이 TILE SIZE - FILTER SIZE + 1이상의 범위를 계산하게 되면, Invalid한 Computation을 하게 된다. 따라서, 이 범위를 넘어가는 threadIdx를 가지는 thread들은 computation을 하지 않도록 하여, 올바른 연산을 보장한다.

추가적으로, 2개의 shared memory 모두에 오른쪽 code와 같이 padding을 하여 Bank Conflict를 방지하였다.

```
const int h_in = blockIdx.y * (TILE_CONV2D_Y - KH + 1) + ty;
const int w_in = blockIdx.x * (TILE_CONV2D_X - KW + 1) + tx;

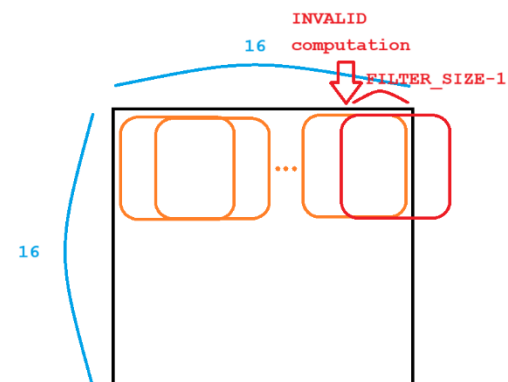
// Load input tile into shared memory
if (h_in >= 0 && h_in < H && w_in >= 0 && w_in < W) {
    X_sh[ty][tx] = X[h_in] * W + w_in;
} else {
    X_sh[ty][tx] = 0.0f;
}

// Load weight tile into shared memory
if (ty < KH && tx < KW) {
    W_sh[ty][tx] = w[ty] * KW + tx;
}

__syncthreads();

// Compute convolution only for valid output threads
if (ty < (TILE_CONV2D_Y - KH + 1) && tx < (TILE_CONV2D_X - KW + 1) && h_out < H_out && w_out < W_out) {
#pragma unroll
    for (int p = 0; p < KH; ++p) {
        for (int q = 0; q < KW; ++q) {
            acc += X_sh[ty + p][tx + q] * W_sh[p][q];
        }
    }
    __syncthreads();

    // Store result
    y[(n * K_out + k) * H_out + h_out] * W_out + w_out] = acc;
}
```



```
_shared_ float X_sh[TILE_CONV2D_Y][TILE_CONV2D_X + 1]; // Input tile
_shared_ float W_sh[MAX_FILTER_SIZE][MAX_FILTER_SIZE + 1]; // Weight tile
```

7-2-1. IM2COL

IM2COL계산에서 Shared Memory를 사용할 때도 마찬가지로 FILTER SIZE로 인해 겹치는 부분을 주의해야 한다. 이번 실습에서 Direct Convolution과 IM2COL의 차이점은 단지 Filter와 Window의 MAC 연산이 im2col에서는 Window내 elements들을 1D Vector로 바꿔서 store하는 것으로 바뀐 것이기 때문이다. 결국, Direct Convolution과 IM2COL 모두 Input Matrix에 대한 Memory access pattern은 동일하다는 뜻이다.

따라서, 오른쪽 code를 보게 되면, shared memory를 사용할 때, 1) TILE 위치 지정, 2) global memory에서 shared memory로의 Load, 3) shared memory내의 element를 이용한 연산 모두 Direct Convolution에서와 indexing이 동일한 것을 확인할 수 있다.

```
// compute output coordinates
const int h_out = blockIdx.y * (TILE_IM2COL_Y - KH + 1) + ty;
const int w_out = blockIdx.x * (TILE_IM2COL_X - KW + 1) + tx;

// load input tile into shared memory
for (int c = 0; c < C; c++) {
    X_sh[ty][tx] = X[h_out] * W + w_out;
}

__syncthreads();

// compute im2col for only valid output threads, considering filter size
if (ty < (TILE_IM2COL_Y - KH + 1) && tx < (TILE_IM2COL_X - KW + 1) && h_out < H_out && w_out < W_out) {
    for (int p = 0; p < KH; ++p) {
        for (int q = 0; q < KW; ++q) {
            int filter_length = KH * KW;
            int C_base = c * filter_length; // base index for filter
            int y = C_base + p * KW + q; // filter index

            // read data from shared memory and store it in output matrix
            X_col[y] * (N * H_out * W_out) + h_out * W_out + w_out] = X_sh[ty + p][tx + q];
        }
    }
}
```

마찬가지로, im2col kernel의 shared memory에서도 padding을 하여 conflict를 방지하였다.

7-3. How about GEMM kernel?

Matrix Multiplication Kernel의 경우 강의안과 동일하게 구성하였다. 강의안에 기재되어 있는 code를 보게 되면, shared memory를 사용하고 있고, load 및 store 과정 모두 coalescing이 지켜지고 있다. 따라서, 이번 MGP강의에서 배운 shared memory 사용으로 인한 temporal locality 향상, coalescing으로 인한 spatial locality 향상이 모두 이루어지고 있다. 즉, 강의안에 소개된 최적화 기법은 모두 적용되어 있기에 따로 수정하지 않고 사용하였다.

```
_global__ void MatrixMulKernel(float* M, float* N, float* P, int Width)
{
    1. __shared__ float subTileM[TILE_WIDTH][TILE_WIDTH];
    2. __shared__ float subTileN[TILE_WIDTH][TILE_WIDTH];

    3. int bx = blockIdx.x; int by = blockIdx.y;
    4. int tx = threadIdx.x; int ty = threadIdx.y;

    // Identify the row and column of the P element to work on
    5. int Row = by * TILE_WIDTH + ty;
    6. int Col = bx * TILE_WIDTH + tx;
    7. float Pvalue = 0;

    // Loop over the M and N tiles required to compute the P element
    8. for (int m = 0; m < Width/TILE_WIDTH; ++m) {
        // Load shared memory phase
        9. subTileM[ty][tx] = M[Row*Width + m*TILE_WIDTH+tx];
        10. subTileN[ty][tx] = N[(m*TILE_WIDTH+ty)*Width+Col];
        __syncthreads(); // Wait for the copy to finish
        // Compute phase
        12. for (int k = 0; k < TILE_WIDTH; ++k)
            13. Pvalue += subTileM[ty][k] * subTileN[k][tx];
        14. __syncthreads(); // Wait for the compute to be complete
    15. }
    16. P[Row*Width+Col] = Pvalue;
}
```

8. Reasons why your program is performing at its best and proving whether performance margin is there or not

8-1) direct convolution & im2col 최적화 분석

5번 및 6번에서 알아본 optimization 가능한 point는 Shared Memory를 사용하여, Direct Convolution Kernel과 Im2Col Kernel의 performance를 개선하는 것이었다.

7번을 통해 이 최적화를 어떻게 구현하였는지 알아보고, 강의안에서 배운 최적화 기법 (shared memory, coalescing, preventing bank conflict)를 모두 적용하였다. 따라서, 현재 code는 강의안에서 배운 내용으로는 더 이상 최적화가 불가하다고 판단을 내렸다.

하지만, 최적화를 진행한 이후 code의 실행속도 성능은 거의 하나도 향상되지 않았다.

추가적으로 최적화하기 위한 기법(thread의 register file을 더 활용하는 방법, im2col에서 filter 1개 안의 $kH \times kW$ element를 1개의 thread가 옮기는 것이 아닌, 1개의 thread가 1개의 element만 load & store하는 방법 등)을 시도하였지만, 이 또한 유의미한 성능 개선이 존재하지 않았다.

따라서, 이 이상 code의 성능개선은 되지 않았고, 8번 문항의 나머지 part에서는 왜 성능 향상이 존재하지 않았는지에 대한 이유를 분석해본다.

오른쪽 code는 Shared Memory를 이용하였을 때, Direct Convolution에서 연산하는 과정과, im2col에서 input Matrix의 element를 column-wise Matrix로 옮기는 과정을 보여준다.

파란색 네모 박스를 보게 되면, 연산에는 모든 threads를 이용하지 않고, FILTER SIZE만큼 제한 수의 threads만 사용되고 있는 것을 확인할 수 있다. 따라서, 연산에 사용되지 않는 threads는

```
[Direct Convolution]
// Compute convolution only for valid output threads
if (ty < (TILE_CONV2D_Y - kH + 1) && tx < (TILE_CONV2D_X - kW + 1) && h_out < H_out && w_out < W_out) {
    #pragma unroll
    for (int p = 0; p < kH; ++p) {
        for (int q = 0; q < kW; ++q) {
            acc += X_sh[ty + p][tx + q] * W_sh[p][q];
        }
    }
}

[im2col]
// compute im2col for only valid output threads, considering filter size
if (ty < (TILE_IM2COL_Y - kH + 1) && tx < (TILE_IM2COL_X - kW + 1) && h_out < H_out && w_out < W_out) {
    for (int p = 0; p < kH; ++p) {
        for (int q = 0; q < kW; ++q) {
            int filter_length = kH * kW;
            int C_base = c * filter_length; // base index for filter
            int y = C_base + p * kW + q; // filter index

            // read data from shared memory and store it in output matrix
            X_col[y * (N * H_out * W_out) + h_out * W_out + w_out] = X_sh[ty + p][tx + q];
        }
    }
}
```

IDLE상태로 아무것도 하지 않기 때문에, 성능 저하를 일으킨다.

만약, 연산에 이용되지 않는 threads를 줄이거나 전부 utilize할 수 있다면, 이에 의한 성능 향상이 있을 것으로 예상된다.

더불어, shared memory를 사용하게 될 시 __syncthreads를 사용하여야 한다. 하지만, __syncthreads는 모든 thread가 synchronization 되도록 마지막으로 연산되는 thread까지 기다렸다가 다음 작업을 시행하는 함수로 상당히 성능을 저하시킨다.

8-2) GEMM 최적화 분석

GEMM kernel의 경우 강의안에 기재되어 있는 code가 shared memory 및 coalescing이 모두 적용되어 있기에 따로 수정하지 않고, 그대로 사용하였다. 추가로, Hardware Specification을 보게 되면, 128, 256, 512 threads per ThreadBlock일 때, occupancy가 100%가 되기에 최대 성능을 보여준다. 하지만, 해당 kernel을 실행시켰을 때는 64 threads per ThreadBlock에서의 성능이 256의 threads를 사용하였을 때의 성능보다 더 좋게 나온다. 이에 대한 이유가 Bank Conflict 때문인 것 같아 이를 개선해보려고 하였으나, Bank Conflict 때문에 발생한 성능 저하는 확인해본 결과 아니었다. 따라서, 이 성능 저하의 이유를 정확히 파악하고, 해결한다면, 256 threads를 이용하였을 때 성능 개선이 존재할 것으로 예상된다.

9. Evaluation including tables or graphs

Not Using shared Memory Execution Time (TILE_X = 16, TILE_Y = 16)

실행 순서	3x3 Direct (ms)	3x3 im2col (ms)	3x3 GEMM (ms)	7x7 Direct (ms)	7x7 im2col (ms)	7x7 GEMM (ms)
1	0.259	0.279	0.568	0.25	1.115	1.761
2	0.257	0.283	0.558	0.249	1.114	1.761
3	0.254	0.287	0.555	0.251	1.114	1.759
4	0.259	0.288	0.553	0.252	1.111	1.76
AVG	0.25725	0.28425	0.5585	0.2505	1.1135	1.76025

Using shared Memory Execution Time (TILE_X = 16, TILE_Y = 16)

실행 순서	3x3 Direct (ms)	3x3 im2col (ms)	3x3 GEMM (ms)	7x7 Direct (ms)	7x7 im2col (ms)	7x7 GEMM (ms)
1	0.259	0.293	0.52	0.278	1.152	1.761
2	0.256	0.282	0.566	0.276	1.147	1.76
3	0.246	0.281	0.567	0.278	1.145	1.761
4	0.246	0.283	0.57	0.28	1.149	1.759
AVG	0.25175	0.28475	0.55575	0.278	1.14825	1.76025

Using shared Memory Execution Time (TILE_X = 32, TILE_Y = 16)

실행 순서	3x3 Direct (ms)	3x3 im2col (ms)	3x3 GEMM (ms)	7x7 Direct (ms)	7x7 im2col (ms)	7x7 GEMM (ms)
1	0.248	0.282	0.566	0.239	1.113	1.759
2	0.244	0.288	0.556	0.238	1.114	1.759
3	0.247	0.28	0.567	0.238	1.115	1.759
4	0.271	0.283	0.557	0.237	1.116	1.761
AVG	0.2525	0.28325	0.5615	0.238	1.1145	1.7595

Using shared Memory Execution Time (TILE_X = 16, TILE_Y = 8)

실행 순서	3x3 Direct (ms)	3x3 im2col (ms)	3x3 GEMM (ms)	7x7 Direct (ms)	7x7 im2col (ms)	7x7 GEMM (ms)
1	0.236	0.285	0.567	0.388	1.035	1.759
2	0.253	0.281	0.569	0.387	1.04	1.761
3	0.248	0.283	0.556	0.388	1.042	1.759
4	0.239	0.281	0.565	0.387	1.035	1.75
AVG	0.244	0.2825	0.56425	0.3875	1.038	1.75725

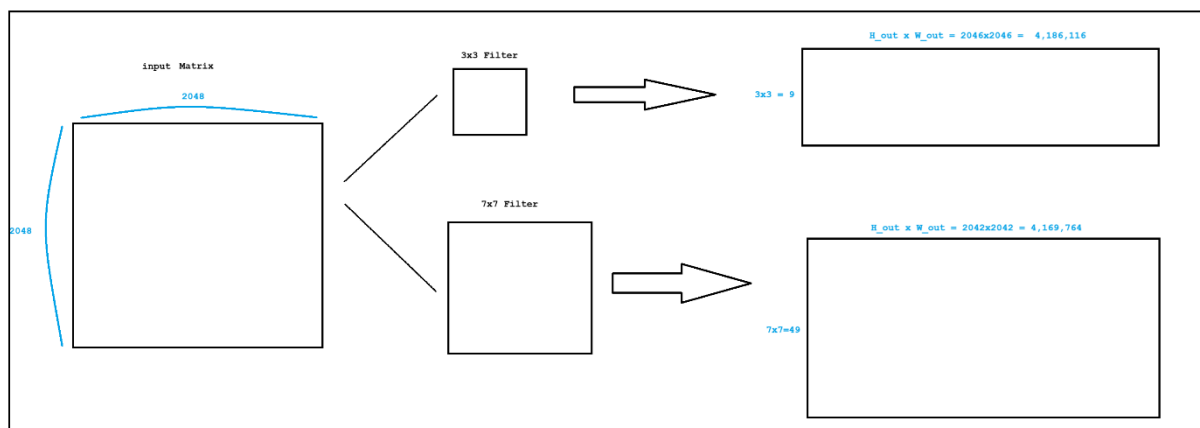
Hardware Specifications에서 RTX 3090은 Threads per ThreadBlock이 128, 256, 512일 때 occupancy가 100%로 최적의 성능을 보여주는 것을 알았다.

따라서, Evaluation은 다음과 같이 진행한다.

- a) shared memory를 사용하지 않은 것
- b) shared memory 사용 & 128 threads per ThreadBlock
- c) shared memory 사용 & 256 threads per ThreadBlock
- d) shared memory 사용 & 512 threads per ThreadBlock

Table에서 빨간색으로 표기한 부분은 4개의 실행 결과 중 각 part에서 최고의 속도를 보여준 결과이다. 따라서, 가장 좋은 performance를 보여준 것은 128 threads per ThreadBlock이지만, 이 경우에는 7x7 Direct의 성능이 0.3ms를 넘기기에 이번 실습의 성능 평가 기준에 미치지 못한다. 이러한 이유로, 모든 요소가 가장 안정적인 성능을 보여주는 512 threads per ThreadBlock을 최종 code로 결정하게 되었다.

10. Why the im2col+GEMM approach does not show the expected performance in this homework?



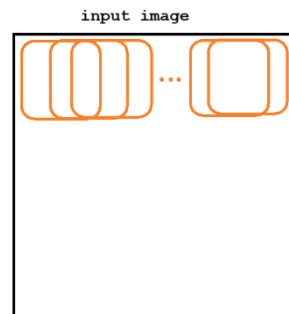
이번 실습의 경우 channel $C = 1$ 이고, 사용하는 filter의 개수 $M = 1$, 사용하는 input의 개수 $N = 1$, stride = 1, dilation = 1, padding = 0으로 진행되었다.

즉, 2048 x 2048 image 1개와 3x3 또는 7x7 filter 1개의 가장 기본적인 2D convolution성능을 측정한 것이다.

더불어, im2col의 이점은 GPU에서 높은 parallelism을 자랑하는 GEMM operation을 이용하여 convolution을 할 수 있다는 것이다.

왜 이번 실습에서는 Direct Convolution에 비해 빠른 속도를 달성하지 못했는지 알아본다.

앞서 설명하였듯이 im2col과 direct convolution의 memory access pattern은 동일하다. 오른쪽 그림에서 각 filter와 겹치는 부분을 im2col은 다시 column-wise하게 재 정렬하는 것이고, direct convolution은 각 filter와 겹치는 부분을 바로 MAC연산을 할 뿐이다.



1) im2col + GEMM의 경우 추가적인 data load가 필요하다.

im2col과 direct convolution 모두 memory access pattern이 동일하지만, im2col의 경우에는 이를 다시 column-wise하게 만든 Matrix Array에 store하고, 다시 load하여 GEMM operation을 진행하고 결과를 store해야 한다. 따라서, 추가적인 load 및 store에 의한 latency가 발생한다.

2) im2col + GEMM은 filter의 개수인 M이 클 때 효과적이다.

앞서 설명하였듯이 input image에 대한 im2col과 direct convolution의 memory access pattern은 동일하다. 하지만, im2col은 Filter는 재정렬하지 않는다. 따라서, filter의 개수가 많을 경우 image를 column-wise를 단 1번 재정렬 후 여러 filter에 대해 GEMM operation을 진행할 경우 GPU에서 GEMM을 할 경우 발생하는 성능의 이점을 충분히 살릴 수 있다.

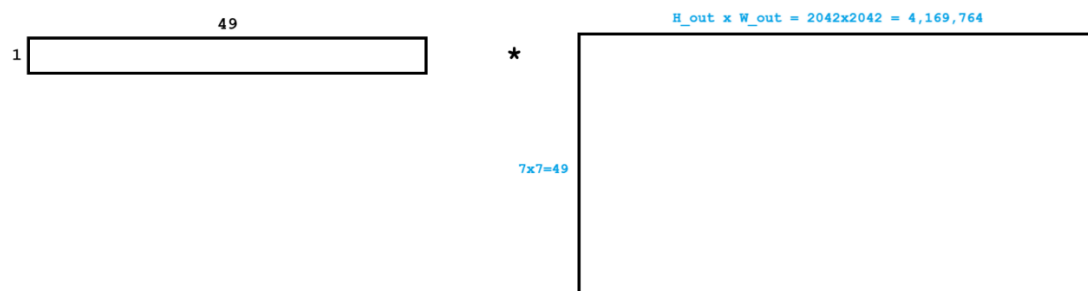
하지만, 이번 실습의 경우, $M = 1$ 이었기 때문에, 성능의 이점이 없고 오히려 direct convolution에 비하여 더 좋지 않은 성능을 보였다.

더불어, 아래 그림을 보면 사실 이번 실습은 GEMM이 아닌 GEMV연산을 한다고 보는 것이 올바르다. 따라서, GEMM 연산에 인한 성능향상을 기대하기 어렵다.

3) im2col은 N과 C의 크기가 클 때 효과적이다.

im2col+GEMM은 본질적으로 대규모의 GEMM을 수행한다. 하지만, 이번 실습의 경우 N과 C가 모두 1이었기 때문에, 아래 그림과 같은 input Matrix가 N, C가 1이 아닐 때보다, 상대적으로 크지 않다. 따라서, GEMM연산으로 인한 속도 향상이 두드러지지 않는다.

다른 말로 표현하자면, im2col은 input Matrix를 큰 Column-wise Matrix로 변환하는 과정에서 Memory Overhead를 발생시키는 데, 이를 상쇄할 만큼 GEMM에서 진행되는 parallel한 연산량이 많아야 한다. 하지만, 이번 실습의 경우 GEMM에서 진행되는 parallel한 연산량이 충분히 많지 않았다.



11. References

- 4_cuda_mgp_2025.pdf
- 5_cuda_matmul_mgp_2025.pdf
- 6_Transpose_bankConflict_mgp_2025.pdf
- 7_cuda_dnn_mgp_2025_merged.pdf